Section A:

Architecture short explanation

The proposed architecture follows a cloud-native, domain-oriented, and loosely coupled approach, designed to support multi-country scalability, resilience, and progressive legacy modernization.

The solution is organized into several layers:

- **Digital Channels + BFF Layer:** centralized web, mobile, and partner experiences through specialized BFFs, optimizing API composition and channel-specific adaptations.
- **Edge and Security Layer:** exposes services through an API Gateway, WAF, and centralized security policies.
- **Service Mesh (Istio + Envoy):** provides cross-cutting capabilities such as circuit brakers, retries, mTLs, and bulkheads isolation, improving resilience and observability.
- **Microservices Layer – Bounded Contexts:** microservices are organized by business domains and deployed on Kubernetes with contry-level isolation.
- **Legacy Integration Gateway:** acts as the single-entry point to legacy systems using the ACL (Anti-Corruption Layer) pattern, decoupling modern domains from legacy contracts and protocols.
- **Data and Sync Integration Layer:** combines decentralized persistence, Redis distributed cache, and event-driven platform based on Kafka with Outbox + CDC to ensure consistency and prevent dual-write issues.
- **Observability Layer:** implements end-to-end tracing with OpenTelemetry, metrics collection through Prometeus, and centralized visualization in Grafana.

The architecture priorizes:

- Resilience
- Horizontal scalability
- Asynchronous integration
- Observability
- And progressive evolution from legacy systems toward a modern event-driven ecosystem.

12-week roadmap

Six sprints × three workstreams (Reliability · Integration Modernization · Observability/Operations).

Sprint 1 (W1–2) — Foundations and Governance

- **Reliability**: SLO/SLI catalog for top 10 endpoints; Global timeout/retry standards; Mesh baseline policies (Istio).
- **Integration Mod**: Inventory legacy integrations (criticality × volume × pain); Domain/context mapping; Dependency critical matrix.
- **Obs/Ops**: Centralized logging standard.
- **Governance**: Architecture governance process.

Sprint 2 (W3–4) — Fast Stabilization

- **Reliability**: Resilience4j framework deployed in 2 pilot services; first chaos test (kill-pod).
- **Integration**: Outbox + CDC pilot in 1 service; AsyncAPI specs adopted.
- **Obs/Ops**: Golden signals dashboards in Grafana; Prometheus alerting; SLO burn-rate alerts; CI/CD reusable pipelines.

Sprint 3 (W5–6) — Pattern rollout

- **Reliability**: Idempotency keys in write APIs (Redis); bulkheads per dependency; Adaptative rate limits.
- **Integration Mod**: ACL pattern for 2 critical legacy systems; schema registry mandatory.
- **Obs/Ops**: Structured JSON logs; correlation ID via trace-id; Shared integrations libraries; API Governance automation.

Sprint 4 (W7–8) — Scale & isolation

- **Reliability**: HPA + PDB tuning; multi-AZ verified; first regional failover.
- **Integration Mod**: Event-driven for cross-context mutations (saga orchestration); Event choreography; Deprecate sync legacy webhooks.
- **Obs/Ops**: SLO error budgets in prod; on-call operations; postmortem template; GitOps federations; Multi-country deployment templates.

Sprint 5 (W9–10) — Multi-country hardening

- **Reliability**: Chaos engineering program (latency injection, DB failover, dep degradation).
- **Integration Mod**: Cross-country contract testing (Pact); APIM federation.
- **Obs/Ops**: Business KPIs in Grafana per country; tail-based trace sampling.

Sprint 6 (W11–12) — Operate & handover

- **Reliability**: Full DR exercise (one country); production readiness checklist mandatory for new services; Reliability audits.
- **Integration Mod**: Legacy deprecation roadmap (6–12 months); Integration fitness functions.
- **Obs/Ops**: SRE practices documented; capacity planning model; cost observability per country; Platform KPI

Section B y C:

Solution Path: sura-tech-lead-assessment/integration-framework

See the README.md for view full detail.

Section D:

ADR-001 — Centralized Integration Platform vs. Decentralized Team-Owned Integrations

Status

Accepted

Context

A multi-country Digital Direct Channel with bounded contexts and per-country cells requires outbound integrations to legacy systems, payment gateways, and third-party APIs. Without a shared approach, each team independently implements timeout, retry, circuit breaker, and observability logic, producing divergent patterns, observability gaps, and inconsistent on-call language across the platform.

Options considered

- **Pure centralized** — Pro: uniform enforcement. Con: delivery bottleneck, ownership dilution, single point of decay.
- **Pure decentralized** — Pro: full autonomy. Con: duplicated resilience code, observability gaps, no shared SLO language.
- **Hybrid — centralized framework, decentralized ownership** — Platform team publishes a versioned SDK (timeout, retry, CB, idempotency, OTel). Domain teams own their integration code and consume the SDK. Pro: uniform primitives without bottleneck. Con: Platform team is critical-path for SDK evolution.

Decision

Adopt the hybrid model. The Platform team owns the integration framework, schema registry, ACL conventions, and observability stack. Each domain team owns its integrations using the framework. This eliminates pattern divergence and observability gaps without introducing an ESB-style bottleneck. The SDK is a paved road, not a gated road: teams can contribute and the framework ships independently of any domain release.

Consequences

- **Positive** — uniform resilience across all services; OTel + Micrometer by default; faster onboarding for new integrations.
- **Negative** — breaking SDK changes require coordinated upgrades across all consumers.
- **Follow-ups** — semantic versioning policy; internal contribution guidelines; Backstage template; one-quarter deprecation runway for breaking changes.

ADR-002 — Event-Driven vs. Synchronous Request-Response for Critical Flows

Status

Accepted

Context

Critical flows span multiple bounded contexts (e.g., origination triggering compliance checks, payments updating transaction core). Choosing a single default communication style forces unacceptable trade-offs: pure synchronous coupling creates tight runtime dependencies and cascading failures; pure async introduces eventual consistency everywhere, including latency-sensitive queries. The architecture must serve both sub-100 ms read paths and reliable cross-context state transitions (see MD-04).

Options considered

- **Default synchronous** — Pro: simple, traceable. Con: tight coupling, cascading failures, no replay.
- **Default asynchronous** — Pro: decoupled, replayable. Con: eventual consistency degrades UX on latency-sensitive paths; overkill for ephemeral queries.
- **Per-flow criteria** — sync for reads; async (Outbox + CDC → Kafka) for cross-context mutations. Pro: each flow uses the appropriate model. Con: two patterns to operate; engineers must apply selection criteria consistently.

Decision

Apply per-flow criteria with explicit defaults: synchronous HTTP for idempotent reads, ephemeral quotes, and intra-cell calls with sub-100 ms; asynchronous event-driven (Outbox + CDC) for any mutation that crosses a bounded context, requires audit/replay, or tolerates tail latency above 100 ms. Dual-writes are forbidden; the Outbox pattern is the only sanctioned mechanism for reliable event publication.

Consequences

- **Positive** — cross-context mutations are decoupled and replayable; no cascading failures across bounded contexts; read paths retain low latency; natural audit log for compliance.
- **Negative** — eventual consistency for cross-context reads; higher operational complexity (Kafka, Debezium, DLQ monitoring, saga compensations).
- **Follow-ups** — AsyncAPI specs mandatory for all events; DLQ alerting runbooks; saga choreography conventions; team training on Outbox pattern.